

Trabajo Practico Final — Practico 9

Testing de Software 2024

Índice

Introduccion	2
1. Ejercicio 1.a: completar la implementacion	3
1.1. Errores detectados y corregidos	3
1.2. Convenciones de codigo aplicadas	7
2. Ejercicio 1.b: suite de tests para ThreesBoard	7
2.1. Modelo elegido	7
2.2. Caracteristicas y particiones	8
2.3. Criterio de cobertura	8
2.4. Requisitos de test	8
2.5. Suite construida	9
2.6. Requisitos inviables	9
2.7. Resultados de cobertura con JaCoCo	10
3. Ejercicio 2: tests de modulo/integracion para los movimientos	10
3.1. Objetivo	10
3.2. Modelo de estados por accion	10
3.3. Inyeccion de aleatoriedad determinista	11
3.4. Requisitos de test	11
3.5. Suite construida	12
3.6. Requisitos inviables	13
3.7. Resultados de cobertura con JaCoCo	13
3.8. Resultado global	14
Conclusiones	14

Introduccion

Este documento contiene la resolucion del *Trabajo Practico Final* (Practico 9) de Testing de Software. El practico integra los conceptos vistos a lo largo del cuatrimestre sobre una implementacion (parcial) del juego *Threes* (<http://threesjs.com>).

Clases provistas.

- `ThreesBoard.java`: modela el tablero de 4×4 y determina que movimientos estan habilitados en un estado dado.
- `ThreesTile.java`: modela una pieza individual del tablero. Una pieza es “libre” (valor 0) o tiene un valor valido: 1, 2 o $3 \cdot 2^i$ con $i \in \mathbb{N}$ (es decir, 3, 6, 12, 24, 48, ...).
- `ThreesController.java`: realiza los movimientos del juego (`moveUp`, `moveDown`, `moveLeft`, `moveRight`), gestiona la carga de la siguiente baldosa en el borde opuesto al movimiento y guarda `nextTileValue`.
- `RandomGenerator.java` / `RandomValueGenerator.java`: interfaz y implementacion concreta para la fuente de aleatoriedad. Se aprovecha en los tests para inyectar una secuencia determinista (`SequenceRandomGenerator`).

Reglas del juego (resumen).

- Tablero 4×4 . Cada juego inicia con 9 piezas aleatorias con valores $\in \{1, 2, 3\}$.
- Cada movimiento desplaza *todas* las piezas en la misma direccion a lo sumo un lugar.
- **Combinaciones:** $1 + 2 \rightarrow 3$ (y $2 + 1 \rightarrow 3$); $x + x \rightarrow 2x$ para $x \geq 3$. Las combinaciones $3 + 6$, $6 + 12$, etc. no son validas.
- Por fila/columna se realiza *a lo sumo una* combinacion.
- Despues de un movimiento, una nueva baldosa (valor 1, 2 o 3) entra en alguna posicion del borde opuesto a las filas/columnas modificadas.
- El juego termina cuando no hay movimiento posible.
- **Invariante:** cada pieza con valor n distinto de 1 y 2 cumple $\exists i \in \mathbb{N} : n = 3 \cdot 2^i$.

Estructura del documento.

- **Ejercicio 1.a:** completar la implementacion, reportar y corregir defectos.
- **Ejercicio 1.b:** suite de tests para `ThreesBoard` usando *Caracterizacion del Dominio de Inputs*, con sus requisitos y la suite que los cubre.

- **Ejercicio 2:** suite de tests de modulo/integracion para `moveUp`, `moveDown`, `moveLeft` y `moveRight` con alta cobertura de predicados medida con JaCoCo.

Ejecucion de la suite y de JaCoCo. Desde `tp9/assignment-9-rodeghiero-1`:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -q test
```

1. Ejercicio 1.a: completar la implementacion

1.1. Errores detectados y corregidos

Durante la implementacion se encontraron varios defectos en el codigo provisto. Cada uno se reporta a continuacion con su correccion.

(1) `ThreesBoard.getTile`: validacion incorrecta de la columna

Defecto. La condicion original permitia el indice `col = COLUMNS`, fuera de rango (`col <= COLUMNS`).

Correccion. Se ajustaron las dos comparaciones a estrictas en el extremo superior:

```
public ThreesTile getTile(int row, int col) {
    if (0 <= row && row < ROWS && 0 <= col && col < COLUMNS) {
        return elements[row][col];
    } else {
        throw new IllegalArgumentException();
    }
}
```

(2) `ThreesBoard(int totalTiles)`: sin validacion de rango

Defecto. El constructor no validaba `totalTiles`. Para valores > 16 el `while` interno entraba en bucle infinito (no quedaban celdas libres).

Correccion.

```
public ThreesBoard(int totalTiles) {
    if (totalTiles < 0 || totalTiles > ROWS * COLUMNS) {
        throw new IllegalArgumentException("totalTiles must be in
                                         [0, 16]");
    }
    // ... resto de la inicializacion ...
}
```

(3) ThreesBoard: valor aleatorio inicial incorrecto

Defecto. La inicializacion aleatoria seteaba valores entre 0 y 2, incluyendo el 0 (celda libre), cuando deberia setear 1, 2 o 3.

Correccion. Se cambia por `generator.getRandom(3) + 1` (rango [1,3]).

(4) ThreesBoard: metodos faltantes

Estaban sin implementar:

- `isFinished()`
- `canMoveLeft()`, `canMoveRight()`, `canMoveUp()`, `canMoveDown()`
- `canTilesCombine(ThreesTile, ThreesTile)`
- `computeScore()`

Decision de diseno. `isFinished()` es simplemente el `!canMoveLeft && !canMoveRight && !canMoveUp && !canMoveDown`. Cada uno de los `canMove*` recorre el tablero en su direccion y devuelve `true` si encuentra al menos una celda no libre que pueda avanzar (porque la vecina en la direccion del movimiento esta libre o es combinable). Se factorizo la logica de combinaciones en `canTilesCombine`.

```
public boolean canMoveLeft() {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 1; j < COLUMNS; j++) {
            if (!elements[i][j].isFree()) {
                ThreesTile current = elements[i][j];
                ThreesTile left = elements[i][j - 1];
                if (left.isFree() || canTilesCombine(left,
                    current)) {
                    return true;
                }
            }
        }
    }
    return false;
}

public boolean canTilesCombine(ThreesTile t1, ThreesTile t2) {
    if (t1 == null || t2 == null) {
        throw new IllegalArgumentException("Tiles cannot be null"
            );
    }
}
```

```

    }
    if (t1.isFree() || t2.isFree()) return false;
    int v1 = t1.getValue(), v2 = t2.getValue();
    return (v1 == 1 && v2 == 2)
        || (v1 == 2 && v2 == 1)
        || (v1 >= 3 && v1 == v2);
}

```

canMoveRight, canMoveUp y canMoveDown son simetricos.

computeScore().

```

public int computeScore() {
    int score = 0;
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLUMNS; j++) {
            int v = elements[i][j].getValue();
            if (v == 3) score += 3;
            else if (v > 3) score += (v - 3) * 3;
        }
    }
    return score;
}

```

(5) ThreesTile.isValidValue: validacion correcta del invariante

Implementacion. Un valor v es valido sii: $v \in \{0, 1, 2\}$ o $v = 3 \cdot 2^i$ para algun $i \geq 0$. La verificacion se reduce a chequear que $v/3$ sea potencia de dos:

```

public boolean isValidValue(int v) {
    if (v == 0 || v == 1 || v == 2) return true;
    if (v < 3 || v % 3 != 0) return false;
    int quotient = v / 3;
    return isPowerOfTwo(quotient);
}

private boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

```

El truco $n \& (n-1) == 0$ es la forma clasica de chequear potencia de dos en una operacion.

(6) ThreesTile.setRandomValue: distribucion incorrecta

Defecto. Originalmente `value = randomGenerator.nextInt() % 3` podia producir valores fuera de $\{1, 2, 3\}$ (incluido el 0 o negativos para enteros aleatorios arbitrarios).

Correccion. `value = randomGenerator.nextInt(3) + 1`.

(7) ThreesController: moveUp, moveLeft, moveRight faltantes y moveDown con defecto

Defecto. `moveUp`, `moveLeft` y `moveRight` no estaban implementados. `moveDown`, segun la version inicial, podia duplicar valores al desplazar (no respetaba la regla de *una sola* combinacion por fila/columna).

Diseno de la correccion. Se factorizo el algoritmo de movimiento en helpers privados, todos documentados con JavaDoc:

- `readRow/readColumn`: leen una linea en el orden del movimiento.
- `collapseAndMergeLine`: colapsa los huecos y aplica una sola combinacion por par adyacente.
- `sameLine`: compara dos lineas para saber si hubo cambios.
- `writeRow/writeColumn`: escriben de vuelta en coordenadas del tablero.
- `loadNextTileOnColumns/loadNextTileOnRows`: cargan la nueva baldosa en el borde opuesto a las filas/columnas modificadas.

```
private int[] collapseAndMergeLine(int[] line) {
    int[] compact = new int[line.length];
    int n = 0;
    for (int v : line) if (v != 0) compact[n++] = v;

    int[] merged = new int[line.length];
    int m = 0, i = 0;
    while (i < n) {
        if (i + 1 < n && canValuesCombine(compact[i], compact[i + 1])) {
            merged[m++] = compact[i] + compact[i + 1];
            i += 2;
        } else {
            merged[m++] = compact[i];
            i++;
        }
    }
}
```

```
    return merged;
}
```

Por que asi. Recorrer la linea ya colapsada de izquierda a derecha y avanzar de *dos en dos* cuando hay combinacion garantiza la regla “una sola combinacion por fila/columna”: una vez combinados dos elementos, el indice salta al siguiente par.

(8) pom.xml: incompatibilidad de JaCoCo con JDK reciente

Defecto. jacoco-maven-plugin 0.8.2 fallaba al instrumentar bytecode con JDK 17 (clases con versiones superiores no soportadas).

Correccion. Se actualizo el plugin a 0.8.12.

1.2. Convenciones de codigo aplicadas

- Cada metodo agregado lleva `JavaDoc` explicito (parametros, retornos, excepciones).
- Las validaciones se hacen al inicio del metodo (*fail-fast*) y lanzan `IllegalArgumentException` con mensaje cuando corresponde.
- Las constantes del tablero estan en `ThreesBoard.ROWS` y `ThreesBoard.COLUMNS` (no *magic numbers*).
- La logica de aleatoriedad se concentra en `RandomGenerator`, lo que permite inyectar generadores deterministas en tests.

2. Ejercicio 1.b: suite de tests para ThreesBoard

2.1. Modelo elegido

Se utilizo **Caracterizacion del Dominio de Inputs** para modelar el espacio de pruebas de `ThreesBoard`, ignorando `toString()` y `getRandom()` como permite el enunciado.

2.2. Características y particiones

ID	Característica	Particiones (bloques)
C1	coordenadas (<code>setTile/getTile</code>)	validas [0..3]; invalidas (< 0 o ≥ 4)
C2	cantidad inicial (<code>ThreesBoard(totalTiles)</code>)	validas [0..16]; invalidas (< 0 o > 16)
C3	ocupacion (<code>numberOfSetTiles</code>)	vacio; parcial; completo
C4	tipo de pareja (<code>canTilesCombine</code>)	$1 + 2$; $2 + 1$; $x + x$ con $x \geq 3$; no combinables; n
C5	movilidad (<code>canMove*</code> , <code>isFinished</code>)	sin movs; por huecos; por combinacion; lleno blo
C6	categoría de valor (<code>computeScore</code>)	$\{0, 1, 2\}$; 3 ; > 3

2.3. Criterio de cobertura

Se utiliza **Each Choice Coverage** extendido a representantes razonables de cada bloque en cada característica. En las características con interacción (e.g. `canMove` en cada dirección) se ejercita cada partición en cada dirección.

2.4. Requisitos de test

- **R1.** `setTile/getTile` dentro del rango actualizan y recuperan el valor de la celda. (*C1 bloque valido.*)
- **R2.** `setTile` rechaza coordenadas fuera de rango. (*C1 bloque invalido.*)
- **R3.** `getTile` rechaza coordenadas fuera de rango. (*C1 bloque invalido.*)
- **R4.** `ThreesBoard(totalTiles)` con valor valido setea exactamente esa cantidad de celdas con valores permitidos. (*C2 valido + C3 parcial.*)
- **R5.** `ThreesBoard(totalTiles)` rechaza `totalTiles` invalido. (*C2 invalido.*)
- **R6.** `numberOfSetTiles` cuenta exactamente las celdas no vacias. (*C3.*)
- **R7.** `canTilesCombine` respeta las reglas para los casos validos e invalidos, y rechaza null. (*C4.*)
- **R8.** Tablero vacio: ningun movimiento es posible y esta finalizado. (*C5 “sin movs”.*)
- **R9.** Tablero con huecos: el movimiento por desplazamiento esta detectado en cada dirección. (*C5 “por huecos”.*)
- **R10.** Tablero con piezas combinables: cada `canMove*` detecta la combinacion. (*C5 “por combinacion”.*)

- **R11.** Tablero lleno sin combinaciones: queda finalizado. (*C5 “lleno bloqueado” + C3 “completo”.*)
- **R12.** `computeScore` suma 3 para valor 3 y $(v - 3) \cdot 3$ para $v > 3$. (*C6.*)

2.5. Suite construida

Nombre: `ThreesBoardInputDomainCharacterizationTest`.

Convenciones:

- `@BeforeEach` crea un tablero limpio para garantizar aislamiento.
- Los rechazos parametrizados (R2, R3, R5) usan `@ParameterizedTest` con `@CsvSource`.
- Los casos de combinacion (R7) tambien son parametrizados.
- Los nombres de los tests son auto-explicativos (`setTileRejectsOutOfRangeCoordinates`, `computeScoreFollowsImplementedRules...`).

Mapeo requisito $\rightarrow$ test.

Requisito	Test que lo cubre
R1	<code>setAndGetTileWithinBounds</code>
R2	<code>setTileRejectsOutOfRangeCoordinates</code> (parametrizado)
R3	<code>getTileRejectsOutOfRangeCoordinates</code> (parametrizado)
R4	<code>constructorWithTotalTilesSetsExactAmount</code>
R5	<code>constructorRejectsInvalidTotalTiles</code> (parametrizado)
R6	<code>numberOfSetTilesCountsNonFreeTiles</code>
R7	<code>canTilesCombineMatchesRules</code> + <code>canTilesCombineRejectsNullTiles</code>
R8	<code>emptyBoardHasNoMovesAndIsFinished</code>
R9	<code>movementByGapsIsDetectedPerDirection</code>
R10	<code>movementByCombinationIsDetectedPerDirection</code>
R11	<code>fullBoardWithoutCombinationsIsFinished</code>
R12	<code>computeScoreFollowsImplementedRules</code>

2.6. Requisitos inviables

- **RI1.** Verificar las *posiciones exactas* de las fichas seteadas por `ThreesBoard(totalTiles)` sin inyectar/mokear `RandomGenerator`. **Motivo:** la posicion depende del generador aleatorio; sin un doble determinista el test seria flaky.
- **RI2.** Cobertura exhaustiva de todos los estados del tablero. **Motivo:** explosion combinatoria; con 16 celdas y multiples valores posibles por celda, el espacio de estados es

inabordable. Por eso se aplica particionado por dominio de inputs.

- **RI3.** Verificar uniformidad estadística del generador aleatorio como tests unitarios. **Motivo:** esa propiedad es de naturaleza estadística, no se valida con asserts deterministas en una corrida unitaria.

2.7. Resultados de cobertura con JaCoCo

Sobre `ThreesBoard` (Ejercicio 1.b):

Metrica	Valor
Lineas	85/97 = 87.63 %
Branches	108/124 = 87.10 %
Metodos	12/13 = 92.31 %

Justificacion de las partes no cubiertas. La fraccion no cubierta corresponde principalmente a:

- Ramas defensivas que ya estan ejercitadas indirectamente por la suite (e.g. comparaciones que se cumplen siempre con los valores del dominio elegido).
- `toString()`: explicitamente excluido del modelo segun el enunciado.
- Algunas ramas del constructor aleatorio que dependen de la fuente de aleatoriedad (cf. RI1).

3. Ejercicio 2: tests de modulo/integracion para los movimientos

3.1. Objetivo

Se busca **alta cobertura de predicados** de las acciones `moveUp`, `moveDown`, `moveLeft` y `moveRight` de `ThreesController`, medida con JaCoCo. Un test que efectua *dos movimientos diferentes en secuencia* cuenta como test de modulo.

3.2. Modelo de estados por accion

Cada accion de movimiento es la composicion de cuatro decisiones internas que el modelo captura como cinco estados representativos:

- **M0:** tablero *no* modificable en esa direccion (no se mueve nada; `loadNextTile` no se invoca).

- **M1:** tablero modificable *solo por desplazamiento* (huecos en la direccion).
- **M2:** tablero modificable con *combinacion* $1 + 2$ o $2 + 1$.
- **M3:** tablero modificable con *combinacion* $x + x$ para $x \geq 3$.
- **M4:** carga de *nueva baldosa* en el borde opuesto con la fila/columna elegida (pseudo)aleatoriamente.

3.3. Inyeccion de aleatoriedad determinista

Como las acciones de movimiento llaman a `generator.getRandom(...)` para elegir posicion de la nueva baldosa y para regenerar `nextTileValue`, se inyecta una `SequenceRandomGenerator` (clase privada del test) que devuelve una secuencia preconfigurada de enteros. Eso permite asserts exactos sobre la posicion final y el siguiente valor cargado.

```
private static class SequenceRandomGenerator implements
RandomGenerator {
    private final int[] sequence;
    private int index;
    SequenceRandomGenerator(int... sequence) { this.sequence =
        sequence; this.index = 0; }
    @Override public int getRandom(int bound) {
        if (bound <= 0) throw new IllegalArgumentException("bound
            must be positive");
        if (index >= sequence.length) return 0;
        int value = sequence[index++];
        return Math.floorMod(value, bound);
    }
}
```

3.4. Requisitos de test

- **R2.1.** Constructor por defecto: tablero inicial con 9 baldosas y `nextTileValue` valido en $[1, 3]$.
- **R2.2.** Si el tablero no admite cambios en una direccion, el movimiento retorna `false` y *no* cambia ni el tablero ni `nextTileValue`. Se parametriza en las cuatro direcciones (M0).
- **R2.3.** `moveLeft` combina, desplaza y carga la nueva baldosa en el borde derecho (M1 + M3).
- **R2.4.** `moveRight` con combinacion $2 + 1$ y carga en el borde izquierdo (M2).

- **R2.5.** moveUp combina y carga en la fila inferior (M3).
- **R2.6.** moveDown combina y carga en la fila superior (M3 + M4).
- **R2.7.** Test de modulo: dos movimientos diferentes en secuencia se comportan consistentemente (M2/M3 + verificacion de estado global).

3.5. Suite construida

Nombre: ThreesControllerModuleIntegrationTest.

Maapeo requisito → test.

Requisito	Test que lo cubre
R2.1	defaultConstructorCreatesExpectedInitialState
R2.2	noChangeMoveKeepsBoardAndNextTile (parametrizado, 4 direcciones)
R2.3	moveLeftCombinesShiftsAndLoadsNextTileOnRightEdge
R2.4	moveRightCombinesTwoPlusOneAndLoadsNextTileOnLeftEdge
R2.5	moveUpCombinesAndLoadsNextTileOnBottomRow
R2.6	moveDownCombinesAndLoadsNextTileOnTopRow
R2.7	sequentialDifferentMovesBehaveConsistently

Ejemplo de test concreto. Para moveLeft:

```
@Test
@DisplayName("moveLeft: combina, desplaza y carga nueva baldosa
en borde derecho")
void moveLeftCombinesShiftsAndLoadsNextTileOnRightEdge() {
    load(board, new int[][] {
        {0, 1, 2, 0},
        {3, 3, 0, 0},
        {6, 12, 0, 0},
        {0, 0, 0, 0}
    });
    board.generator = new SequenceRandomGenerator(
        1, // constructor: nextTileValue = 2
        1, // moveLeft: elige movedRows[1] => fila 1
        0 // nuevo nextTileValue = 1
    );
    ThreesController controller = new ThreesController(board);

    boolean moved = controller.moveLeft();

    assertTrue(moved);
}
```

```

assertBoardEquals(new int[][] {
    {3, 0, 0, 0},    // 1+2 = 3, queda en columna 0
    {6, 0, 0, 2},    // 3+3 = 6; baldosa nueva (valor 2) en
                     // borde derecho
    {6, 12, 0, 0},   // sin combinacion: 6 y 12 no se combinan
    {0, 0, 0, 0}
}, board);
assertEquals(1, controller.getNextTileValue());
}

```

Comentarios.

- La secuencia inyectada permite asserts *exactos* sobre la posición de la baldosa nueva.
- El test ejercita simultaneamente M1 (desplazamiento del 1 al 0), M2 ($1 + 2$) y M3 ($3 + 3$).
- La fila 2 (6, 12, 0, 0) demuestra que las piezas con $x \neq y$ y $x \geq 3$ no se combinan.

3.6. Requisitos inviables

- **RI2.1.** Comprobar la *distribucion estadística* de la baldosa nueva usando aleatoriedad real. **Motivo:** no determinista; en este nivel se trabaja con `SequenceRandomGenerator`, dejando la propiedad estadística fuera del scope de integración reproducible.
- **RI2.2.** Cobertura exhaustiva de todas las secuencias posibles de estados del juego. **Motivo:** explosión combinatoria; se aplica particionado representativo del dominio (M0-M4).

3.7. Resultados de cobertura con JaCoCo

Sobre `ThreesController` (Ejercicio 2):

Metrica	Valor
Instrucciones	596/596 = 100.00 %
Branches	71/78 = 91.03 %
Lineas	124/124 = 100.00 %
Metodos	19/19 = 100.00 %

Justificación de las branches no cubiertas (7 sobre 78). Las ramas no cubiertas corresponden a casos defensivos como:

- Comprobaciones simétricas en `loadNextTileOnColumns` y `loadNextTileOnRows` para movimientos sin cambios (`movedColumns` o `movedRows` vacíos) que ya quedan cubiertos

por el flujo principal.

- Algun camino interno del helper `canValuesCombine` que es alcanzable solo con combinaciones que no aparecen en las configuraciones representativas elegidas (por ejemplo, $a == 1 \wedge b \neq 2$ con $b = 0$).

Se opto por no agregar tests sintetizados solo para forzar cobertura de esas ramas, ya que no aportan deteccion adicional y la consigna pide *justificar* en lugar de inflar artificialmente la suite.

3.8. Resultado global

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -q test
```

- Total de tests ejecutados: **38** (entre ambas suites).
- Fallos: **0**.
- Errores: **0**.

Conclusiones

- El trabajo cierra el ciclo completo del curso: *leer codigo*, *completar la implementacion* (con deteccion y correccion de defectos), *modelar* el espacio de pruebas con una tecnica formal (caracterizacion del dominio de inputs) y *verificar* la suite con cobertura medida por una herramienta externa (JaCoCo).
- La separacion entre `ThreesBoard` (modelo + reglas) y `ThreesController` (acciones) hizo natural dividir el practico en dos niveles: tests *unitarios* sobre el tablero y tests *de modulo/integracion* sobre el controlador.
- La inyeccion de `RandomGenerator` como dependencia permitio escribir asserts exactos sobre acciones que internamente son aleatorias. Sin esa inyeccion, varios requisitos habrian quedado en la categoria de “inviabiles”.
- Los requisitos marcados como inviabiles (RI1-RI3 y RI2.1-RI2.2) son intencionalmente honestos: explicitan los limites de un test unitario o de integracion deterministico y separan lo que puede testearse de lo que requeriria otro tipo de analisis (estadistico o combinatorio).